

Modelling the Water Level of a Glacier Stream using Meteorological Data

Theresa Hayek / Hans Hahne

July 17, 2026

Contents

1	Context and Scope of the Project	3
1.1	Special Characteristics of Hydrological Systems	3
2	Data Description	4
2.1	Presentation of the Raw Data	5
3	Project Workflow / Summary	10
4	Data Preprocessing	13
4.1	Preprocessing of the water level (Target Variable)	13
4.2	Timestamp Handling	13
4.3	Missing Values	13
4.4	Data Quality Checks	16
5	Exploratory Data Analysis	17
5.1	Correlation Analysis	17
5.2	Lag Analysis with rolling covariance analysis	17
5.2.1	Temperature	18
5.2.2	Atmospheric pressure, Radiation, Wind speed, Precipitation and Snow height	18
5.3	Seasonal Effects	21
6	Feature Engineering	24
6.1	Feature Engineering for Machine Learning Models	24
6.1.1	Time averaging	24
6.1.2	Lag Feature	24
6.1.3	Snow Memory	24

6.1.4	Cyclic yearly feature	24
6.2	Feature Engineering for Deep Learning Models	25
7	Model Application	27
7.1	Model Application of Machine Learning Models	27
7.1.1	Linear Regression	27
7.1.2	Autoregressive models: Prophet and SAMIRAX	32
7.1.3	Lazypredictor classify best ML models	34
7.1.4	Results for HbGBR	37
7.2	Model Application of Deep Learning Models	38
8	Conclusion	43
8.1	Conclusion on Machine Learning Models	43
8.1.1	Dynamic behavior of the models	44
8.1.2	Autocorrelation Models (Prophet and SARIMAX) . . .	44
8.2	Conclusion on Deep Learning Models	45
9	Outlook	45

1 Context and Scope of the Project

The “Bayerische Akademie der Wissenschaften” maintains since 1973 a high alpine environmental observatory around the “Vernagtferner”, a glacier located in the Ötztal Alps in Tyrol, Austria. The “Vernagtferner” nourishes the “Vernagtbach” and the measurement station “Pegelstation Hydrologie” measures the water level of this stream. The closeby measurement station “Pegelstation Meteorologie” measures meteorological variables related to air temperature, atmospheric pressure, humidity, precipitation, snow height, wind and radiation. Both stations are in 2640 m. There is another measurement station called “Schwarzkögele” in a height of 3080 m in a distance of about 1 km from the “Pegelstation Hydrologie” and “Pegelstation Meteorologie”. There also exist measurement stations on the ice of the glacier, but because of the harsh weather conditions they cannot always deliver continuous data.

The objective of this data science project is to analyze the feasibility of predicting the water level of the “Vernagtbach” measured by “Pegelstation Hydrologie” from the available meteorological data measured by “Pegelstation Meteorologie” and “Schwarzkögele”. This could be useful to be able to get information about the water level of the “Vernagtbach” without actually measuring it.

The problem of the project is that it is not a conventional data science problem. It is not a typical regression problem because the time dependence spoils the independence between the observations. And it is not a typical time series problem because it does not aim at the prediction of the water level from the previous water levels. This led to difficulties with the definition of what is to be taken as input and output data for a model and what models can be used in which way.

1.1 Special Characteristics of Hydrological Systems

Hydrological systems exhibit several important characteristics:

- Temporal dependencies
- Delayed responses to rainfall and snow melt
- Seasonal effects
- Nonlinear relationships

For example, precipitation does not immediately affect the stream level. Instead, the response may occur several hours later due to infiltration, runoff processes, and storage effects in the catchment.

Therefore, past observations must be incorporated into the predictive model.

2 Data Description

The available data comprises measurements every 5 minutes for 12 years from January 1, 2013 until December 31, 2024 for the measurement stations “Pegelstation Hydrologie” and “Pegelstation Meteorologie” and for over 9 years from July 22, 2015 until December 31, 2024 for the measurement station “Schwarzkögele”. The data has undergone a first preprocessing that removed outliers and corrected for known offsets. It is available as a text file with comma-separated values.

The raw data includes the following measurement data from the 3 measurement stations “Pegelstation Hydrologie”, “Pegelstation Meteorologie” and “Schwarzkögele” and is exclusively numeric:

- “Pegelstation Hydrologie”:
 - water level
from a gauge measurement (water_level)
 - water level
from a measurement with ultrasound (water_level_ultrasound)
- “Pegelstation Meteorologie”:
 - air temperature (PM_temperature)
 - atmospheric pressure (PM_atmospheric_pressure)
 - relative humidity (PM_relative_humidity)
 - precipitation (PM_precipitation)
 - cumulative precipitation (PM_precipitation_cum)
 - snow height (PM_snow_height)
 - wind speed (PM_wind_speed)
 - wind direction (PM_wind_direction)
 - short wave radiation downwards (PM_SWD)
 - short wave radiation upwards (PM_SWU)
- “Schwarzkögele”
 - air temperature (SK_temperature)

- relative humidity (SK_relative_humidity)
- wind speed (SK_wind_speed)
- wind direction (SK_wind_direction)

2.1 Presentation of the Raw Data

The raw data is presented with 3 plots for corresponding variables. One plot shows the data for the whole time period. The other two plots show exemplarily the values for January 1, 2024 and July 1, 2024.

Water Level (See Figure 1) There are no measurements of the water level in winter as the measurement instruments do not work when the air temperature is below zero and snow has fallen. For these times a low constant water level can be safely assumed. The measurement of the level with ultrasound still contains faulty values. There is a pronounced offset between the two measurement methods. The values of water_level_ultrasound are solely to be used to fill the gaps with missing values of water_level.

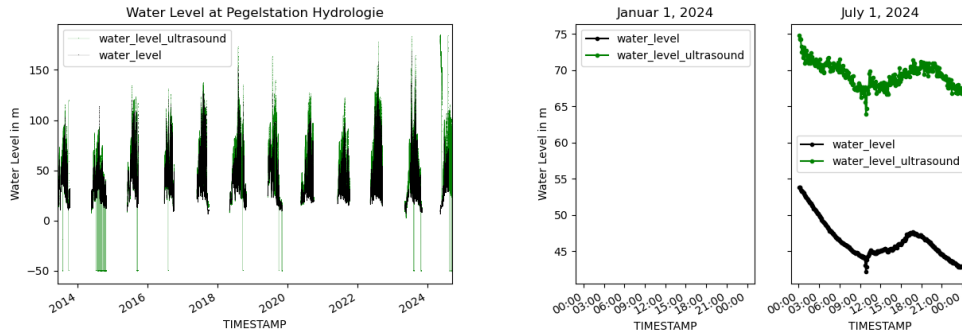


Figure 1: Water Level

Air Temperatures (See Figure 2) The air temperature of Schwarzkögele is not available in the first 2 years and generally lower than the air temperatures at Pegelstation Meteorologie. This is due to the greater height of Schwarzkögele.

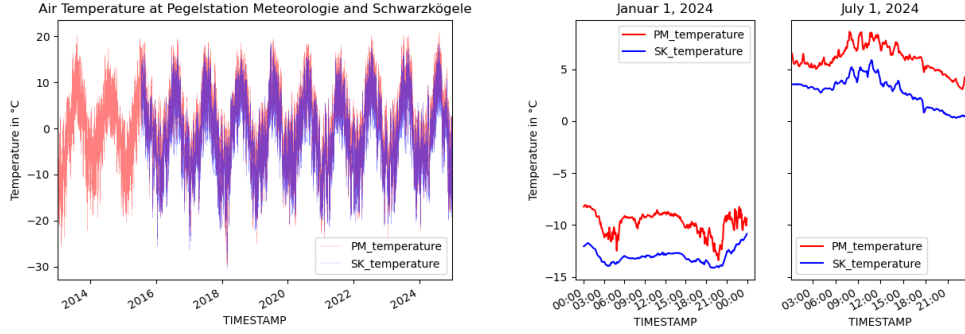


Figure 2: Air Temperature

Atmospheric Pressure The atmospheric pressure at Pegelstation Meteorologie with values around 735 hPa reflects the height of the measurement station 2640 m compared to the mean sea level atmospheric pressure of 1013.25 hPa (See Figure 3)

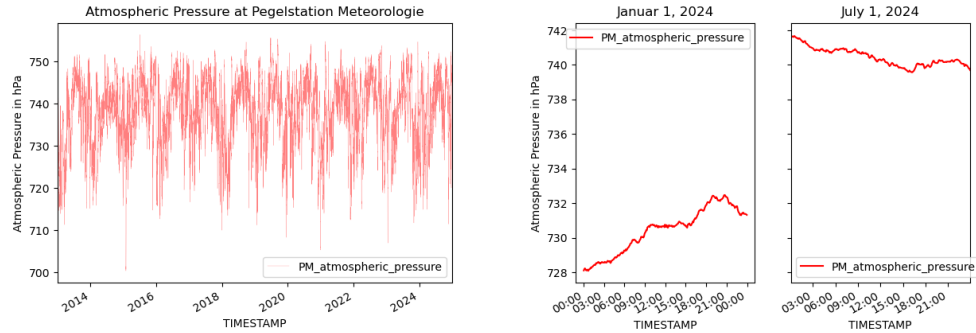


Figure 3: Atmospheric Pressure

Relative Humidity (See Figure 4) The plots for relative humidity show a great difference between the measurement stations Schwarzkögele and Pegelstation Meteorologie. As relative humidity should be in the range between 0 and 100 and SK_relative_humidity shows a different behavior from some time in 2018 SK_relative_humidity has to be regarded invalid and should not be used for modeling.

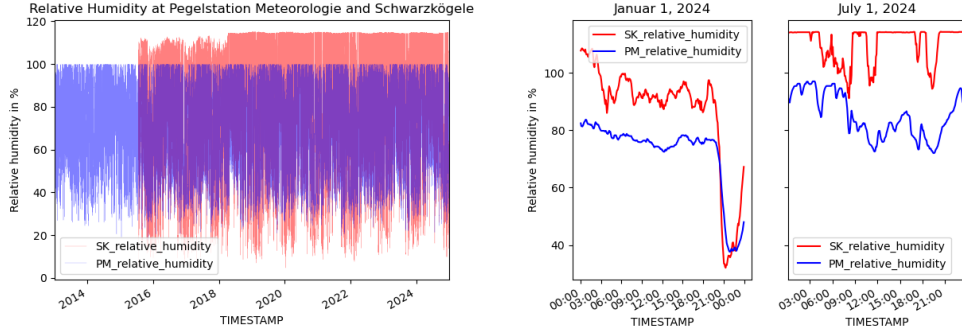


Figure 4: Relative Humidity

Precipitation (See Figure 5) The precipitation `PM_precipitation` measured in $\text{mm}/(5 \text{ min})$ has measurement gaps during the winter in the earlier years because the measurement instrument was then disassembled in winter. The precipitation `PM_precipitation_cum` measured in mm is the cumulative precipitation during a year and is corrupted by the evaporation of water from the measurement instrument. `PM_precipitation_cum` shows a significant measurement gap in 2019. As the precipitation is measured in two ways it should be sufficient to keep one measurement.

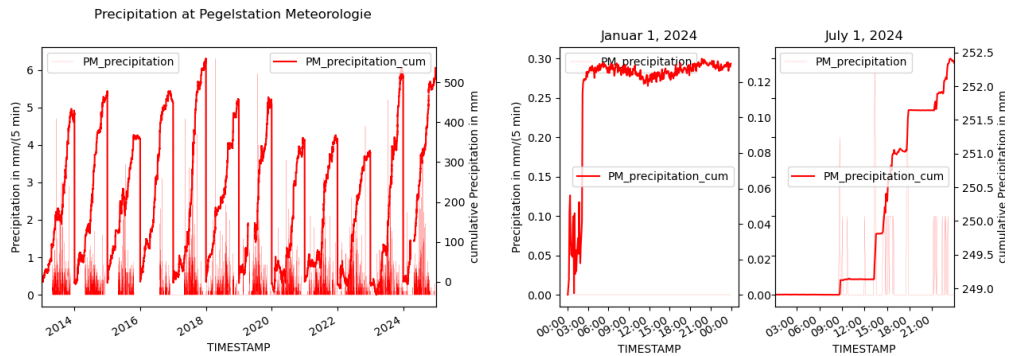


Figure 5: Precipitation

Snow Height (See Figure 6) The snow height exhibits several missing values. However as the snow height does not show larger variations over time it should be safe to interpolate missing values.

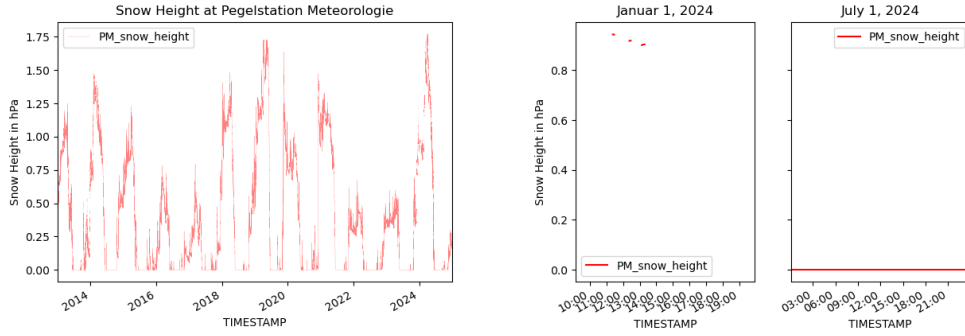


Figure 6: Snow Height

Wind Speed (See Figure 7) The wind speeds measured at Pegelstation Meteorologie and Schwarzkögele show a little offset probably because of the use of different measurement instruments.

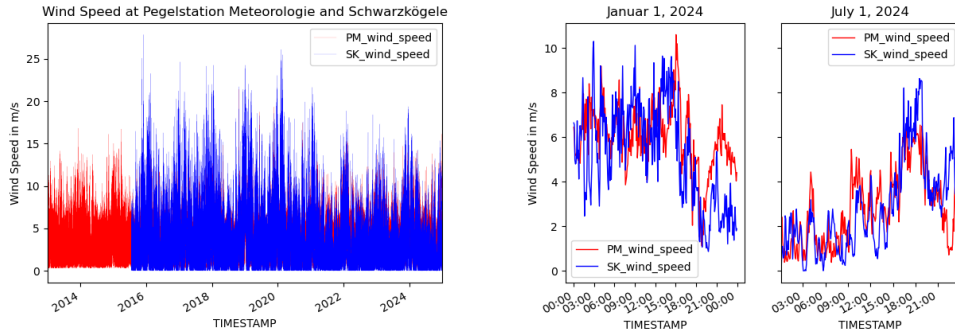


Figure 7: Wind Speed

Wind Direction (See Figure 8) There are 2 main wind directions around 150 and 330 degrees at the measurement stations which are more pronounced at Pegelstation Meteorologie located deeper down the glacier valley.

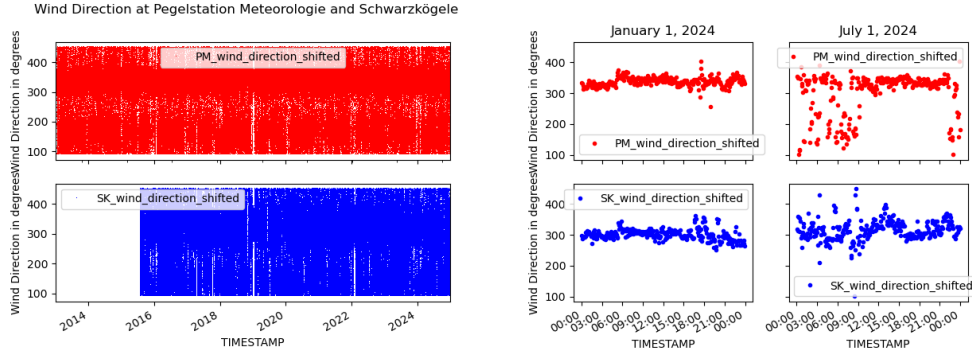


Figure 8: Wind Direction

Short Wave Radiation (See Figure 9) Two kinds of short wave radiation are measured at Pegelstation Meteorologie. The Short Wave Radiation Downward (PM.SWD) is the short wave radiation coming from above mainly from the sun. It is strongest during the summer. The Short Wave Radiation Upward (PM.SWU) is the short wave radiation coming from below. It is mainly part of the short wave radiation from above reflected by the ground. In winter a lot of the short wave radiation from the sun is reflected by snow on the ground but when the snow is gone the Short Wave Radiation Upward is levelling.

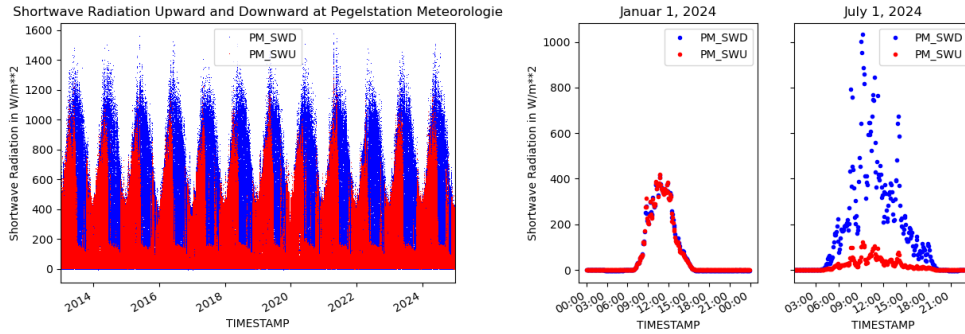


Figure 9: Radiation

3 Project Workflow / Summary

To give an upfront information the following chapter shows how the workflow i.e. the data handling was carried out to generate the predictions for the water level.

Component Description

- **Data_prep.py (Preprocessing)**
 - **Input:** 1. sweep: Raw uncleaned dataset.
Next sweeps: modified data (`mod_meteohydrodata.csv`)
 - **Process:** Fill gaps with data from the past (`shift(xD)`). Remaining gaps (smaller than ca. 6h) filled with `ffill()`.
 - **Output:** `mod_meteohydrodata.csv`.
- **Data_analysis.py (Data Analysis)**
 - **Input:** `mod_meteohydrodata.csv`.
 - **Process:** Exploratory Data Analysis (EDA), Var- and covariance for PM variables, Rolling covariance between water level and variables with high covariance in matrix to identify Lag Variables, Seasonal Effects and Temperature trend analysis with significance test acc. to Kendall.
 - **Output:** Correlation heatmap, Covariance plots, Time plots
- **Feature_Eng.py (Feature Engineering)**
 - **Input:** `mod_meteohydrodata.csv`.
 - **Process:** Encoding is not necessary because DF consist only of measurement data.
 - * Because of computing time requirements the data set is accumulated from 5 min time intervals to daily based time stamp.
 - * Generation of Lag Variables with input from EDA.
 - * Creation of seasonality variables with sinusoidal function for day and year. The yearly seasonality is divided in a constant leg for winter period (no change in water level) and a half sin wave for the summer period.
 - * The variable snow storage is implemented to picture the presence of snow and ice during summer (glacier definition).

- **Output:** `mod_feature_meteohydrodata_hourly.csv` (or split matrices).
- **ML_x.py (Machine Learning)**
 - **Input:** `mod_feature_meteohydrodata_hourly.csv`.
 - **Process:** Following ML models / predictor were investigated:
 - * LinearRegression
 - * Autoregressive models: Prophet and SAMIRA
 - * LazyPredictor (depending on test period length two different ML models were best rated)
 - * GammaRegressor (long test period)
 - * Histogram-based Gradient Boosting Regressor (short test period)
 - **Output:** Comparison of RMSE and R^2 metrics.

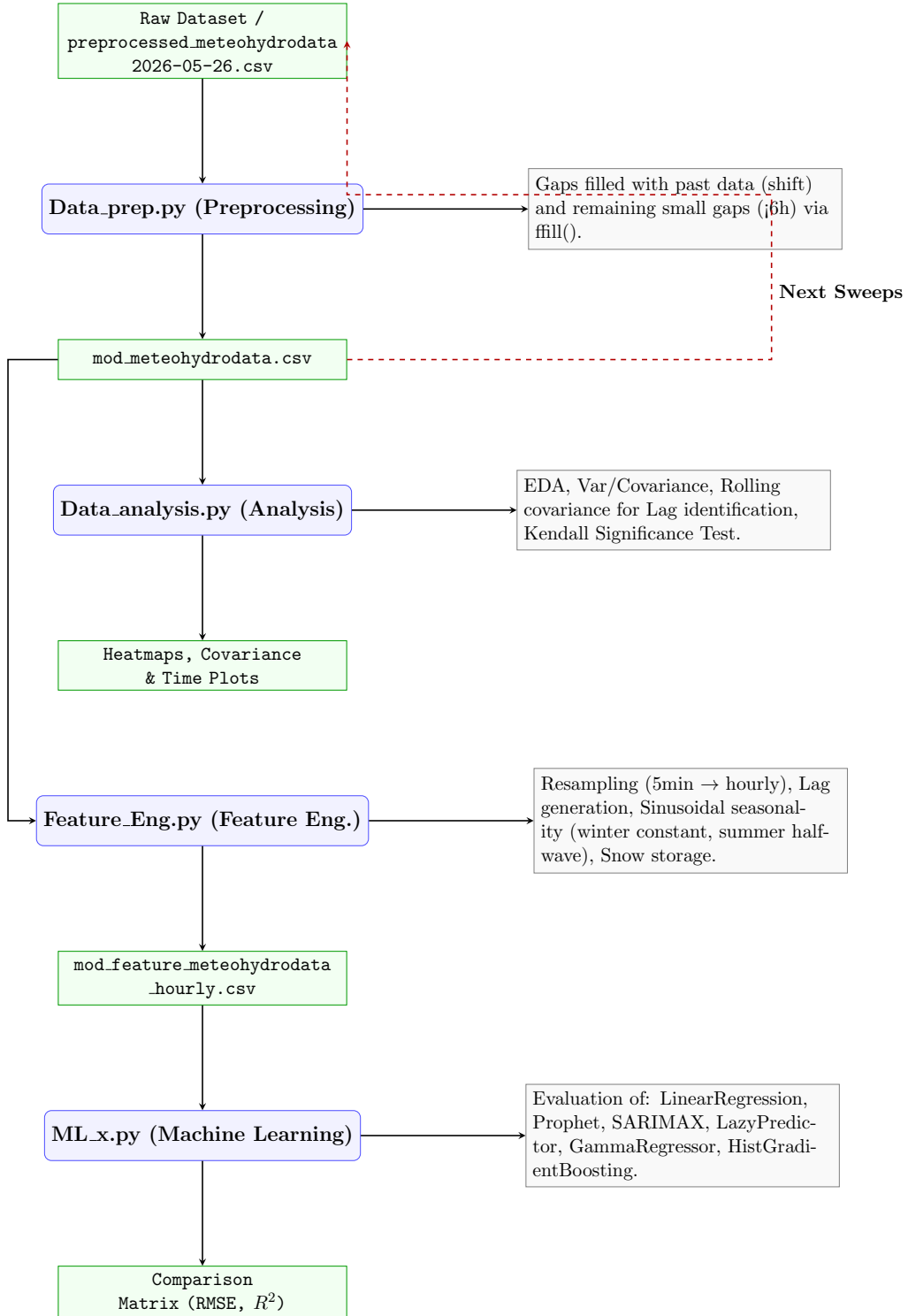


Figure 10: Systematic Data and Modeling Workflow for Water Level Prediction

4 Data Preprocessing

4.1 Preprocessing of the water level (Target Variable)

For short periods of missing values the missing values of `water_level` are interpolated. Longer gaps of missing values of `water_level` are filled by the values of `water_level_ultrasound` that are adjusted to fit the last and first values of non missing values of the `water_level`.

4.2 Timestamp Handling

Convert timestamps to datetime format and sort chronologically.

4.3 Missing Values

For the descriptive variables the two methods:

- Forward shifting
- Forward filling

Were applied.

For Temperature, Atmospheric pressure, Humidity, Wind speed, Wind direction and Solar radiation, missing values, which time period was within $O(\text{one day})$, were filled by data from one or two days before the data gap (Forward shifting). The assumption is that the variables do not change significantly in the considered time interval. This approach is described in the following pictures.

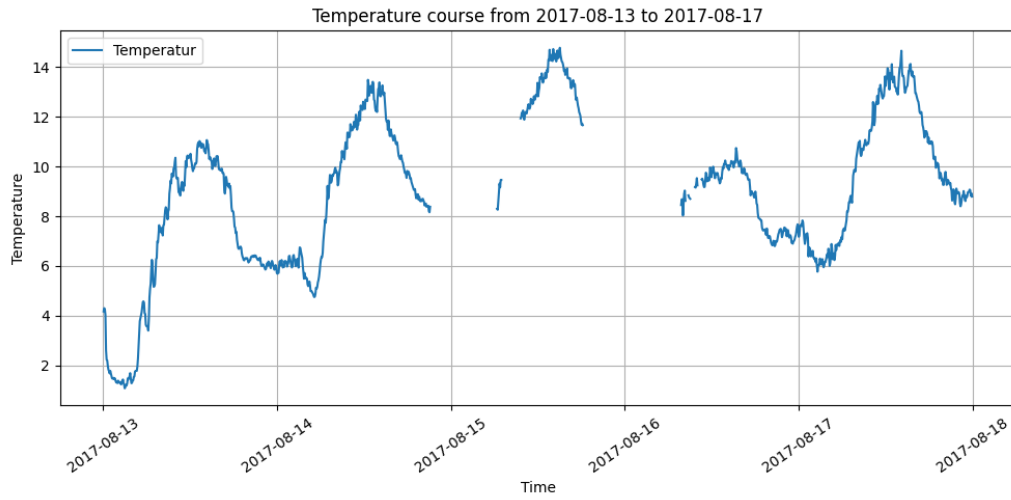


Figure 11: Temperature cutout with gap

The picture shows the raw data distribution of the temperature in the time interval 2017-08-13 to 2017-08-17 with gaps in the temperature course. The biggest gap is from 2017-08-15 18:17:30+00:00 to 2017-08-16 06:42:30+00:00, which was determined by a python script counting nan to find the biggest gaps.

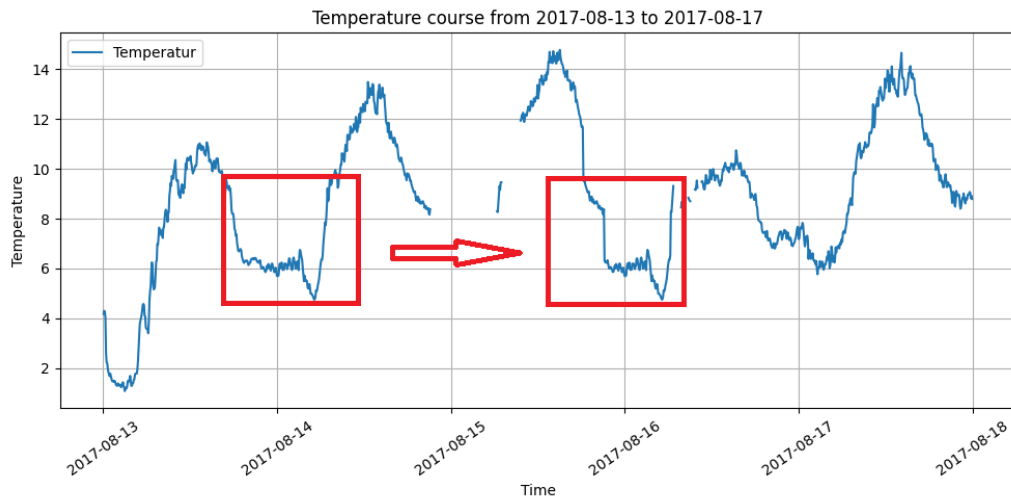


Figure 12: Temperature cutout with closed gap

Like the above figure shows leads this method to discontinuities at the points where the data is connected (current / history data). With the above

assumption the expectation is that this jump is in a tolerable range (e.g. $dT \leq 5$ K, $dP \leq 1$ HPa). For the above example the jump of the temperature is 0.9°C over 2h.

The remaining gaps were filled by forward filling. Final result see picture below.

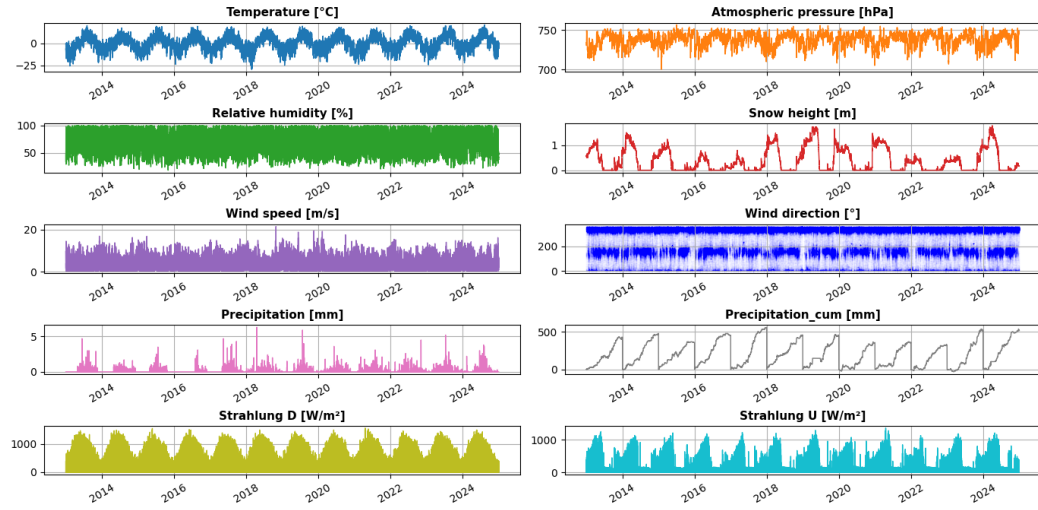


Figure 13: Explanatory Variables

The preprocessed water level is shown in the next picture.

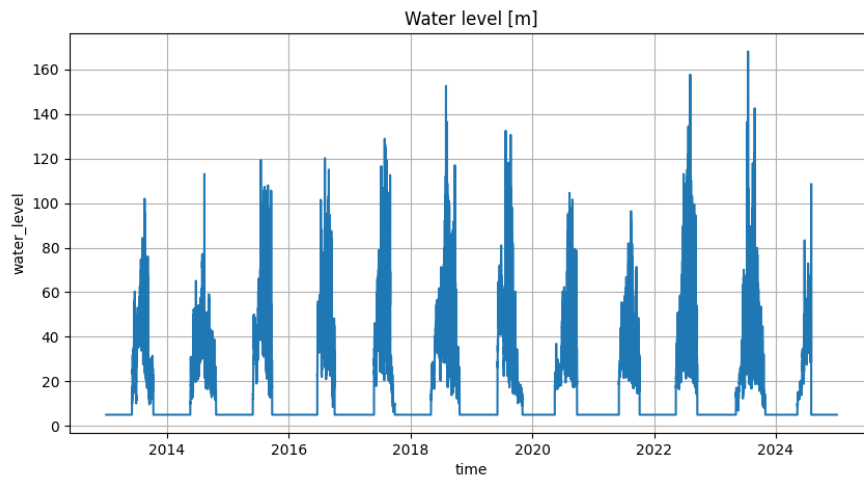


Figure 14: Preprocessed water level

4.4 Data Quality Checks

Perform plausibility checks:

- Negative precipitation values not seen
- Unrealistic sensor jumps not detected

These issues are not detected, see table Data Audit and the following figure.

The variation in the values is due to physical factors, such as actual temperature fluctuations at the measurement site. A time-stable fluctuation should be observable over the measurement time period. The magnitude of the variations depends on the specific measurement variable, see figure below. The variation in the differences between consecutive measurement values should remain within the range of the raw (unmodified) data. The expected value of the measurement differences should be reasonable for 5 min steps and within the limits of the measurement chains' accuracy.

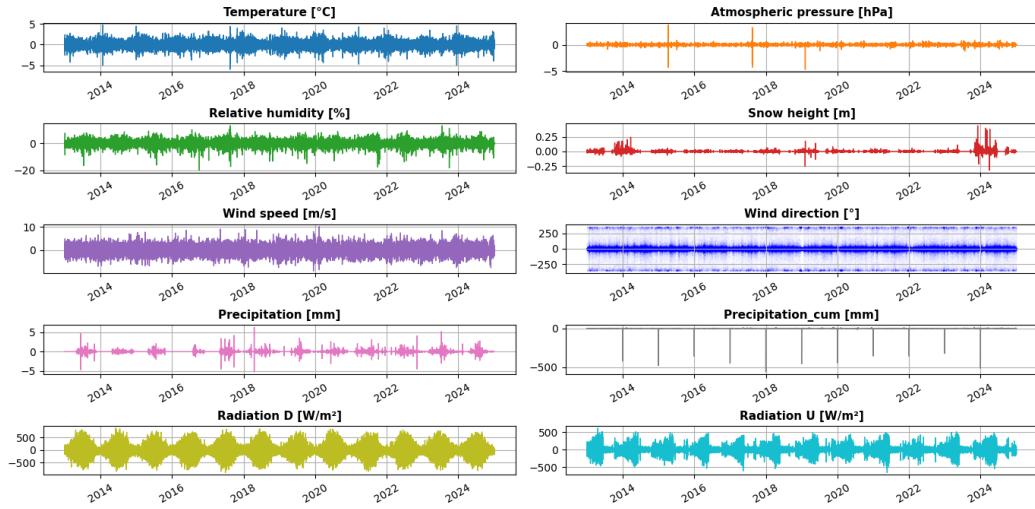


Figure 15: Explanatory Variables with difference, lag = 1 measurement

5 Exploratory Data Analysis

5.1 Correlation Analysis

Initial correlations between variables investigated using correlation matrices (Pearson).

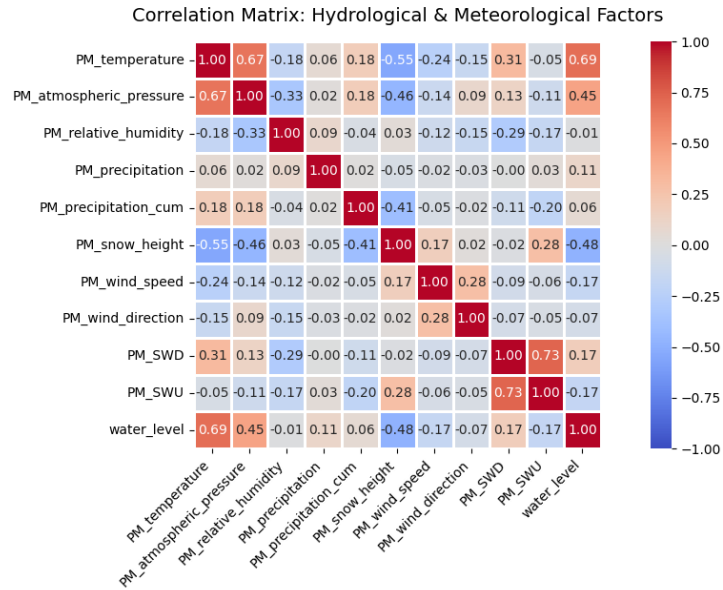


Figure 16: Correlation between all Variables

Based on (0.1 = weak, 0.3 = moderate, 0.5 = strong) from the statistician Jacob Cohen: Cohen, J. (1988). Statistical Power Analysis for the Behavioral Sciences (2nd ed.). Lawrence Erlbaum Associates, it can be seen that 3 variables have a strong influence (Temperature, Atmospheric pressure and Snow height. Wind speed, Percipitation and Radiation have a weak influence.

However, simple correlations may not fully capture delayed hydrological responses.

5.2 Lag Analysis with rolling covariance analysis

Investigate delayed effects between weak predictors and the target variable. The covariance analysis is determent with a time shift of 5 min. for a daily period.

This leads to an insight if there is a lag between explanatory variable and the water level.

5.2.1 Temperature

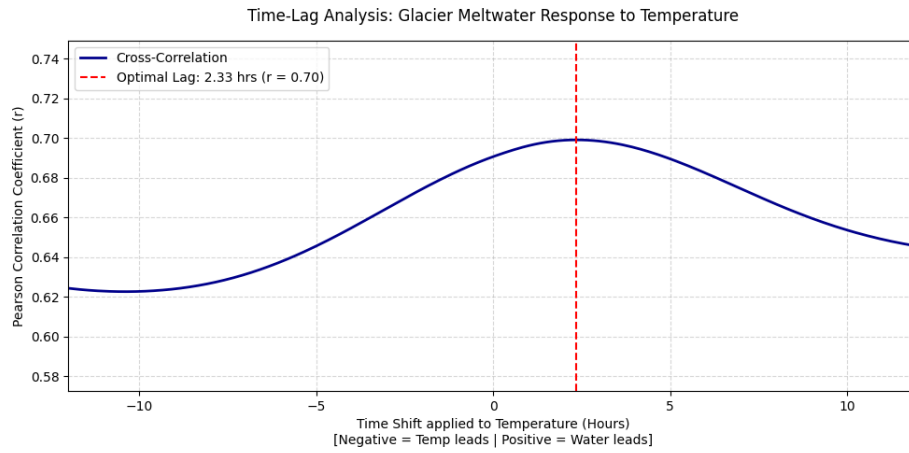


Figure 17: Time lag Temperature / Water level

The analysis shows an offset of 2,3 hours between temperature and water level.

5.2.2 Atmospheric pressure, Radiation, Wind speed, Precipitation and Snow height

Investigate delayed effects between weak predictors (Wind speed, Precipitation and Radiation) and the target variable.

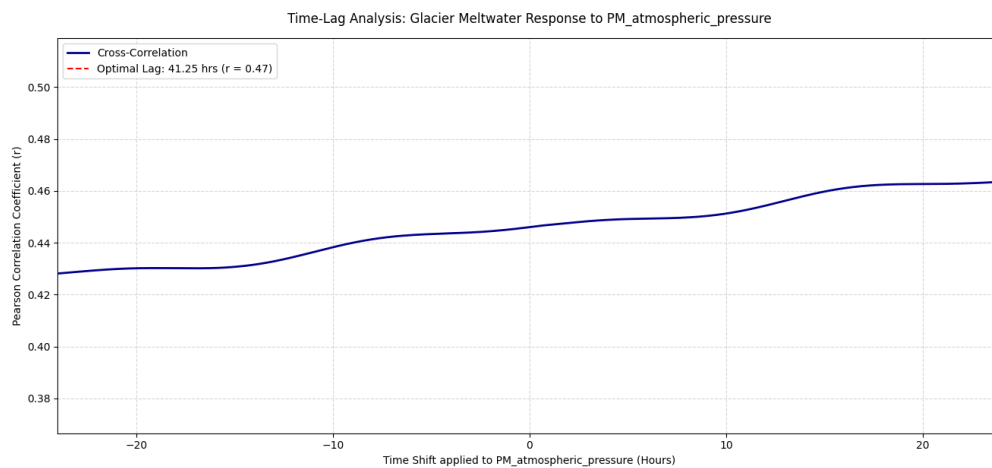


Figure 18: Time lag Atmospheric pressure / Water level

The Atmospheric pressure is an indirect parameter, because there is no direct physical influence on the water level. The indirect influence is due to less clouds at high pressure, this is there is an effect on the radiation.

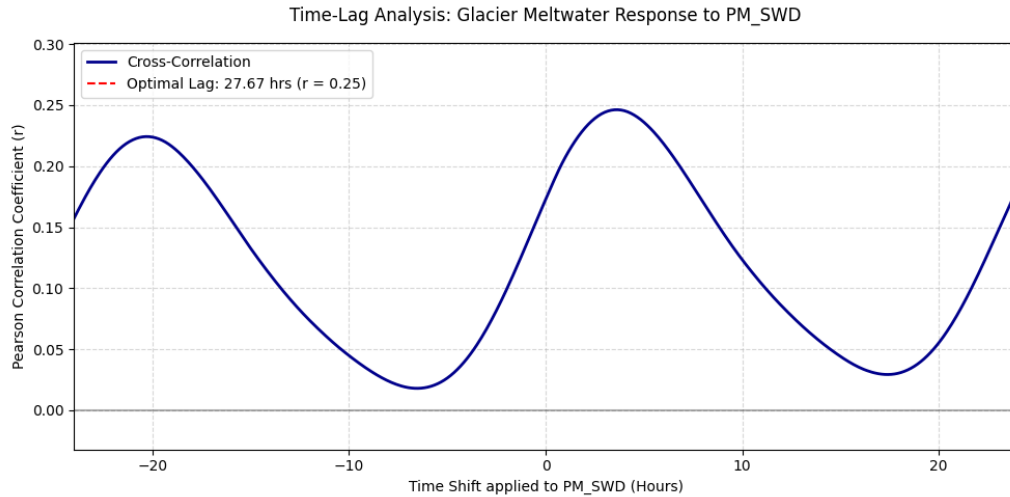


Figure 19: Time lag Radiation down/ Water level

Considering the time lag, the r value for the variable SWDown increases to 0.25. Which changes the influence from weak to moderate.

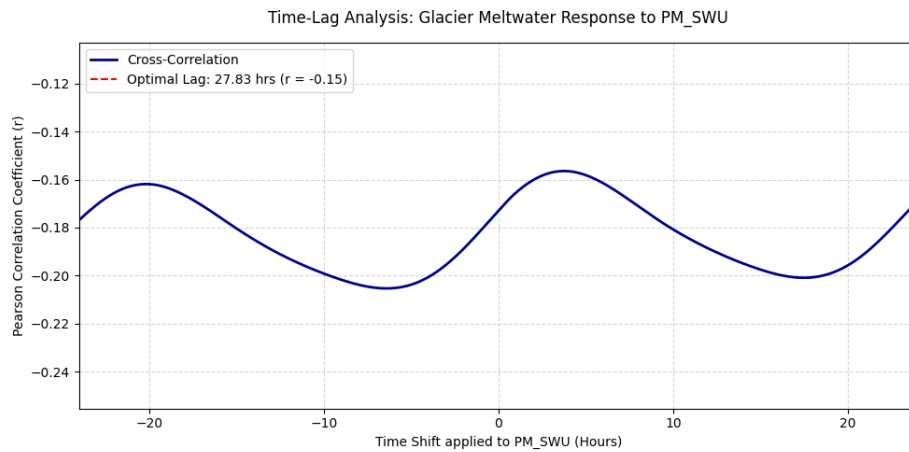


Figure 20: Time lag Radiation up/ Water level

The time lag analysis shows nearly no change of the value $SWU(p)$, this is the influence is still weak.

This behavior is physically coherent. Because due to the SWU radiation the snow lose energy (makes it cooler), due to the SWD radiation it gathers energy(makes it warmer). This is SWD generates potentially drain.

The snow height could have an offset because of the glacier structure (caves or other capacities for storing).

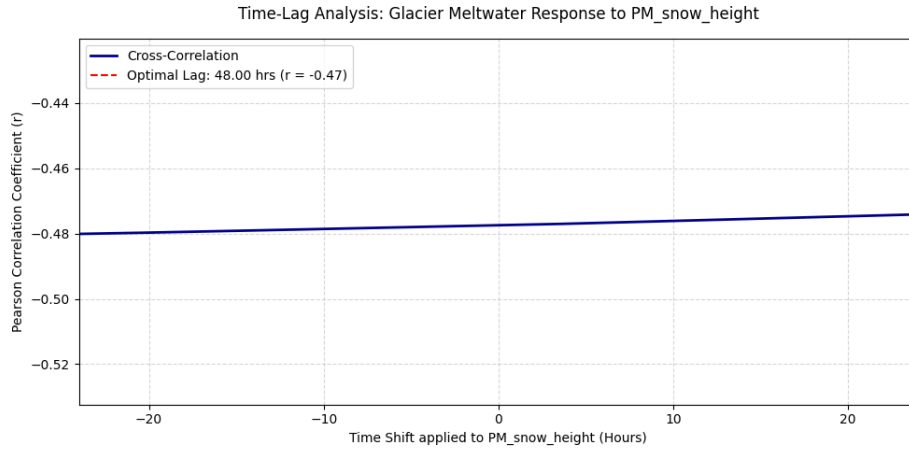


Figure 21: Time lag Snow height / Water level

There is nearly no offset between snow melting and water level. What is reasonable when no capacities present.

5.3 Seasonal Effects

At first the seasonal effects and trends for the target variable are considered.

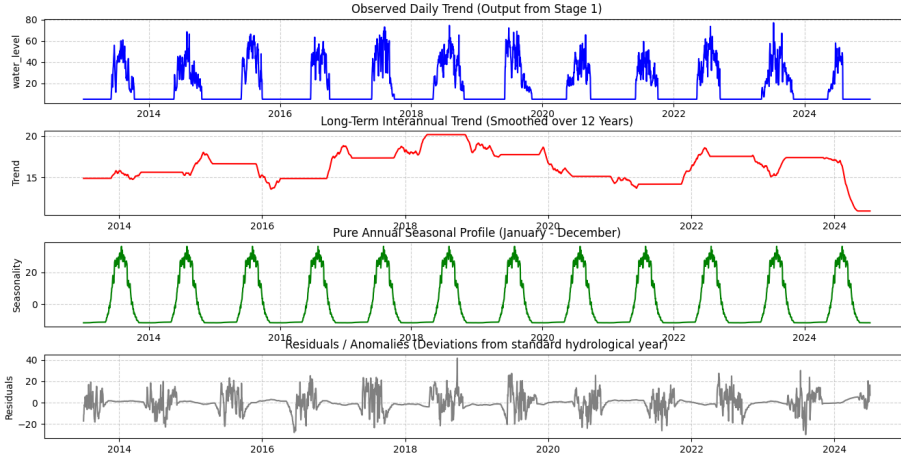


Figure 22: Water level

The trend shows a maximum of the water level in the year 2019. The seasonal effects represent the four seasons of a year.

The most correlated variable temperature is next decomposed.

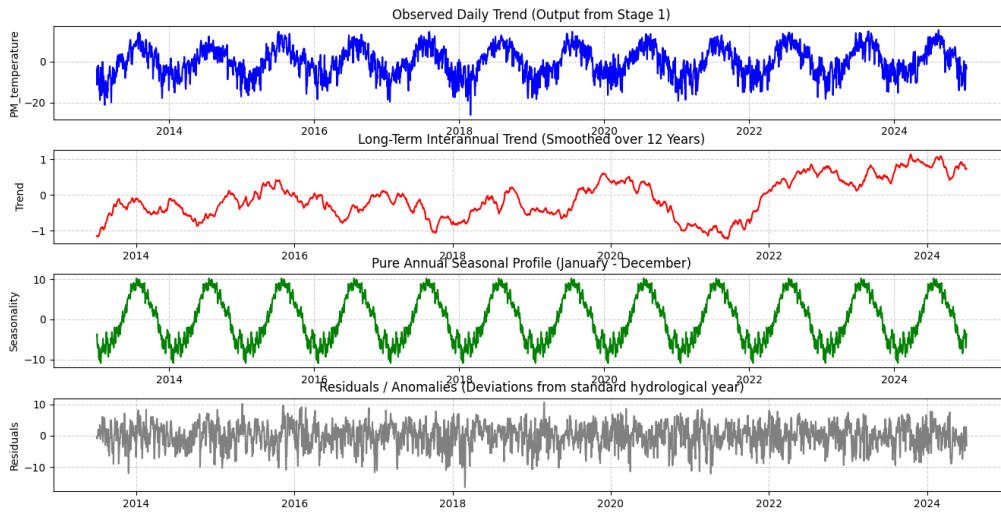


Figure 23: Temperature

The trend shows an increase over time with around 1°C. The seasonality follows the seasons of the year.

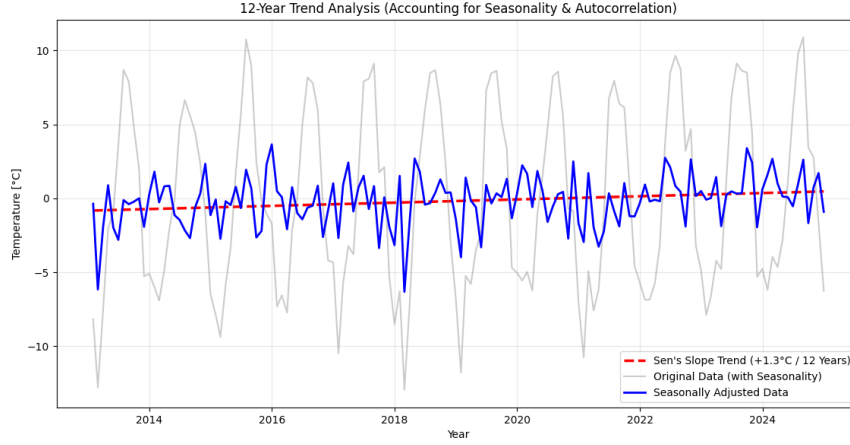


Figure 24: Temperature trend analysis

To identify a scientifically valid long-term trend from the 1.2 million raw temperature data points (PM_temperature), the following three-step statistical workflow was applied: Monthly Aggregation: The high-resolution raw data was resampled to monthly mean values (`.resample('ME')`). This reduced noise and compressed the dataset into exactly 144 consistent data points over the 12-year period. Seasonal Decomposition: An additive seasonal decomposition was performed to isolate the sharp summer-to-winter fluctuations (seasonality). Subtracting this seasonal component yielded the Seasonally Adjusted Data (the baseline trend free from yearly weather cycles). Autocorrelation-Corrected Trend Test: A Correlated Seasonal Mann-Kendall Test was applied to the adjusted series. This specific test accounts for serial dependency (autocorrelation)—the tendency of consecutive months to influence one another. Mathematically, it adjusts (inflates) the variance of the test statistic to ensure that short-term weather phases do not fake a long-term climate trend, see fig. 24.

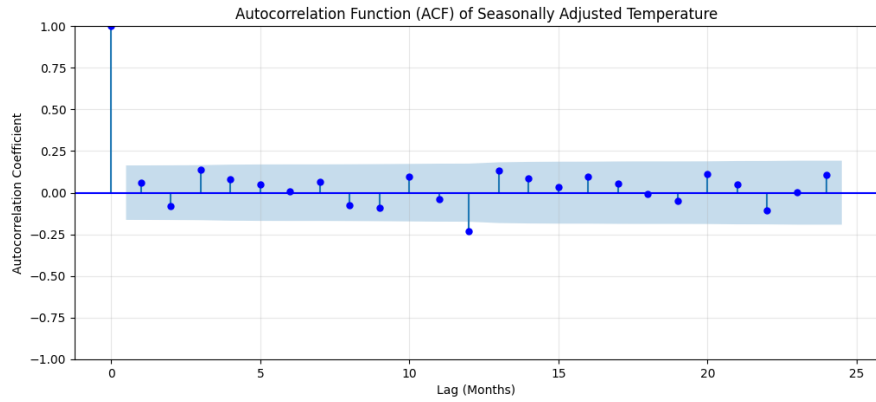


Figure 25: Temperature trend analysis

Trend Quantification: Sen’s Slope was calculated within the test framework to determine the true magnitude of the change. This confirmed a steady, statistically robust temperature increase of approximately 1.3 °C over the entire 12-year period.

Table 1: Climate Trend Significance Report (International Standards)

Metric / Parameter	Value / Analysis Result
Analysis Period	2013-01-01 to 2024-12-31
Total Observations	144 months (Aggregated from 5-min data)
Trend Direction	INCREASING
<i>P</i> -Value (<i>p</i>)	0.035440
Statistical Status	SIGNIFICANT ($\alpha < 0.05$)
Calculated Slope	0.1135 °C per year
Extrapolated Change	+1.36 °C over the 12-year period
Methodological References for Reporting:	
– Trend Significance: Seasonal Kendall Test (Hirsch et al., 1982)	
– Slope Robustness: Seasonal Sen’s Slope Estimator (Gilbert, 1987)	

6 Feature Engineering

6.1 Feature Engineering for Machine Learning Models

6.1.1 Time averaging

The data is aggregated from a 5min frequency to a daily based data set. Result `df_daily`.

6.1.2 Lag Feature

Water Level The water level depends on the water level in the past. To implement that a lag feature for the water level is generated with one day lag.

Result `df_daily['wl_lag1d']` PROBLEM: If it is done in this way, a data leakage is generated for the variable, which should be predicted.

Lead to excellent results, but as the water level for the future is not known, these lag variable can not be calculated for the prediction period.

Temperature, SWD, Wind Speed and Precipitation According to the Data Analysis the moving covariance analysis ($dt = 5min$) for the temperature and the water level shows a maximum at a time shift of ca. 2,3h. This is two lag variables are generated with a time shift of two hours respectively with three hours.

The same is for the other variables (SWD, Wind Speed and Precipitation) with different time shifts for their maximum.

6.1.3 Snow Memory

The snow height measurement stagnates in the spring to fall season. There is still snow and ice on the glacier (that makes a glacier). To estimate this condition the variable snow storage is generated. It takes the last snowfall height as a capacity with is reduced over the year when the temperature on the PM station is above zero degree Celsius.

$storage = storage - melted\ snow.$ $melted\ snow = K \times T_{max}$, $T_{max} \geq 0$ or zero.

6.1.4 Cyclic yearly feature

According to the Data Analysis a yearly seasonality was implemented with a maximum in summer and a constant value between October and April.

The maximum in the summer is continuously generated with a sin-function.
Result `df_daily['summer_wave']`

6.2 Feature Engineering for Deep Learning Models

The input for Deep Learning Models has to be different from the input for Machine Learning Models because of the different nature of Machine Learning Models and Deep Learning Models especially under the circumstances of time series data. Machine Learning Models can produce outputs independent of the length of the input (the length of the time series). Simple Deep Learning Models always take inputs of the same size and return outputs of the same size. This applies also to the length of the time series given that the Deep Learning Model is to output a result dependent on the time of the input time series. The use of a Deep Learning Model is said to catch non-linear relationships in the data so there is no need to set up features regarding time lags. As Deep Learning Models are said to be prone for problems with discontinuous data a variable with an yearly artificial discontinuity is feature-engineered. Having stated this it will follow the feature engineering comprising the creation of a new time variable, the conversion of the wind variables (PM_wind_speed, PM_wind_direction, SK_wind_speed, SK_wind_direction), the conversion of the cumulative precipitation PM_precipitation_cum and the deletion of some variables

Creation of a new time variable During the set up of the input data for a first Deep Learning Model it was a problem that the data loses track of time and there was no way to create a complete time series from the output of the Deep Learning Model. Therefore a new linear variable `time_s_norm` is created that is the time in seconds since 2013-01-01 00:00:00 normalized to be 0 at 2013-01-01 00:02:30 and 1 at 2024-12-31 23:57:30.

Conversion of the wind variables The wind direction in degrees is very variable and quite often there is a jump from just below 360 degrees to just above 0 degrees. By converting PM_wind_speed and PM_wind_direction (SK_wind_speed and SK_wind_direction) to a wind vector PM_wind_speed_X_direction and PM_wind_speed_Y_direction (SK_wind_speed_X_direction and SK_wind_speed_Y_direction) this effect can be eliminated.

Conversion of the cumulative precipitation The cumulative precipitation PM_precipitation_cum measured shows a yearly discontinuity at the

beginning of the year because of the reset of the measurement data of the cumulative precipitation to zero. It is not just a measurement of the precipitation but also the evaporation of water from the measurement instrument. Therefore two new variables are created by first taking the difference between each measurement value and the value one time step before this measurement value and then storing the positive values in a variable precipitation and the negative values in a variable evaporation.

Deletion of variables The variable SK_relative_humidity is deleted because of obvious malfunctioning of the measurement instrument during the whole measurement period. The variable PM_precipitation has pronounced measurement gaps in the beginning of the whole measurement period and measures about the same as PM_precipitation_cum and is therefore deleted. The variable water_level_ultrasound is deleted because it just served to fill the gaps of water_level.

After the splitting in a training dataset, a validation dataset and a test dataset all variables except the new time variable time_s_norm and the target variable water_level are standardized by removing the mean of the training dataset and scaling to the unit variance of the training dataset.

7 Model Application

7.1 Model Application of Machine Learning Models

For all models the train / test split were done explicit by fix dates. After the split the "StandardScaler()" were used. Fitted with the train data and applied to the test data. The performances of the models were checked by RMSE and the Coefficient of determination.

Start with simple and interpretable model.

7.1.1 Linear Regression

This provides a reference performance level. First approach with plain data set, i.e. without implementation of features from the feature engineering (FE) step.

Only data from the Pegelstation Meteorologie (PM) were considered.

Second approach, the features from FE analysis were implemented step by step to evaluate the influence on the model.

- Day and yearly seasonality (via sinusoidal functions)
- Lag features (Temperature, Radiation, Wind speed, Precipitation)
- Snow storage

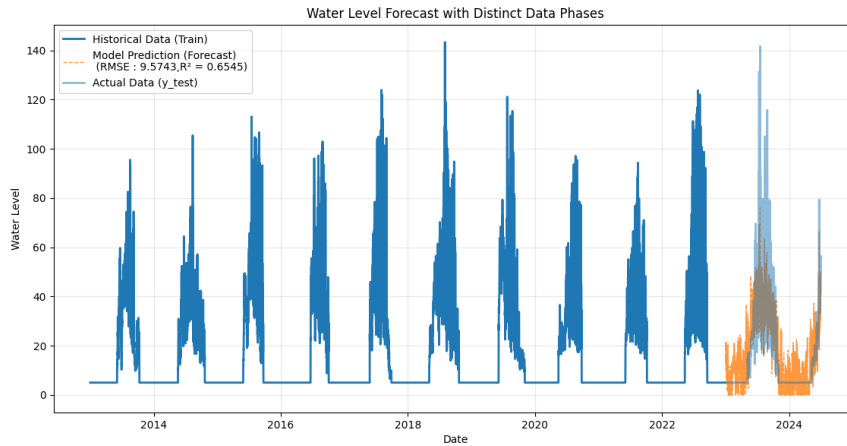


Figure 26: Water level, plain data

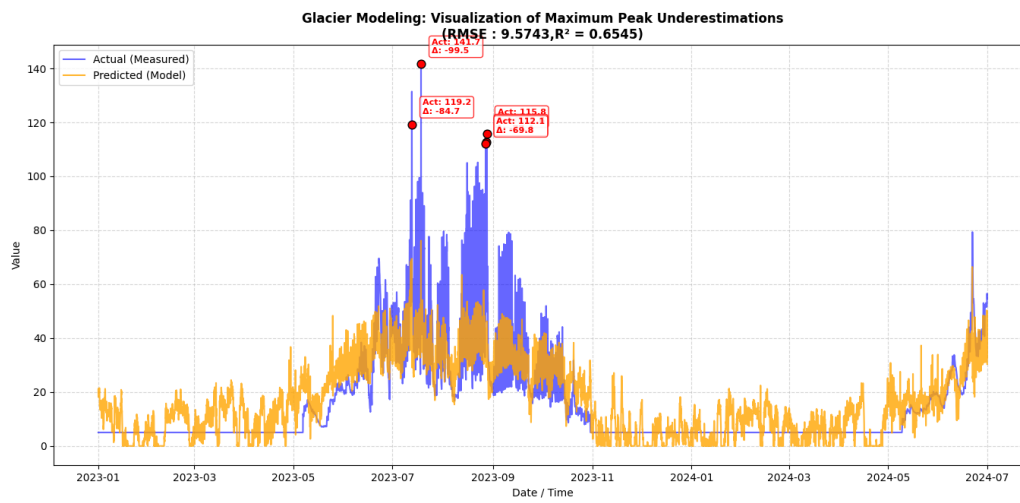


Figure 27: Water level: Test period, plain data

Results Results with all additional features.

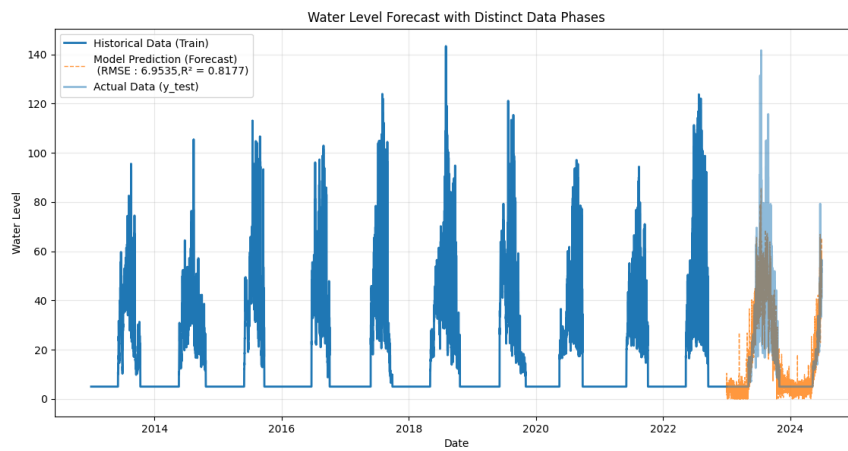


Figure 28: Water level: Test period, with complete FE data

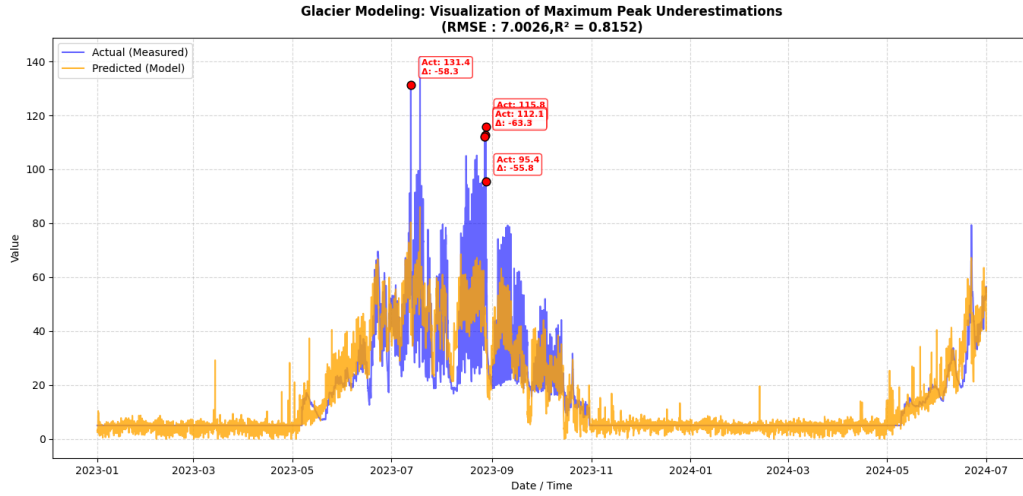


Figure 29: Water level: Test period, with complete FE data

Table 2: Comparison of RMSE and R^2 metrics for different feature configurations, Test data

Configuration / Features	RMSE	R^2
Plain	9.574	0.6545
Seasonality (d/y)	8.498	0.7277
Seasonality (d/y) + Snow storage	7.145	0.8076
Seasonality (d/y) + Snow storage + Lag features	6.9535	0.8177
Seasonality (d/y) + Snow storage + Lag features + SK	7.0228	0.8141

A continuous improvement can be seen when features detected in EDA and FE added to the model.

The error for the training data is higher than the error for the test data. This can be explained by the temporal asymmetry of the data split, which has a 5:1 ratio (a 10-year training period versus a 2-year test period). The much longer training timeframe inherently captures a higher density of meteorological anomalies, unrepresentative extreme events, and systemic noise that distort global fit metrics like the RMSE. Conversely, the shorter 2-year test period exhibits lower variance and fewer extreme shocks, allowing the seasonal and storage-based patterns learned during training to predict the unseen data with higher relative precision.

For comparison an alternative linear regression model is tested. It evaluates the regression errors in a different way, which leads to different outcomes.

GammaRegressor / Results For comparison an other linear ML model is applied to the problem. Difference (main): Residual do along with a gamma distribution.

This assumption has to be checked when deploy the model.

The results shown below are with complete FE generated features.

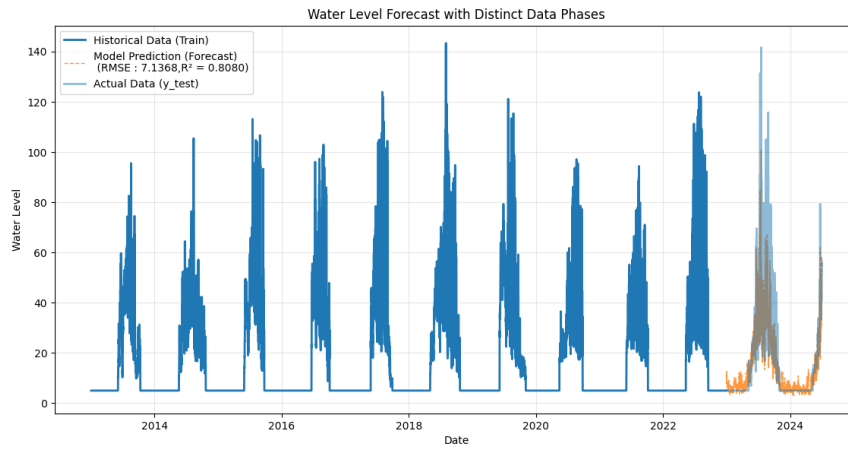


Figure 30: Water level prediction

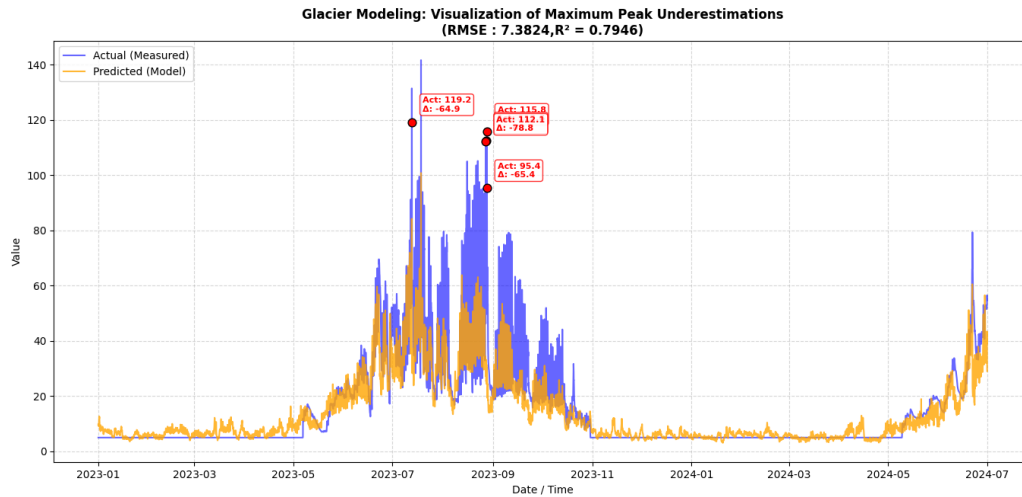


Figure 31: Water level prediction, Prediction detail

Table 3: GammaRegressor RMSE and R2 metrics

Dataset Split	RMSE	R^2
Train	9.8510	0.7187
Test	7.3824	0.7946

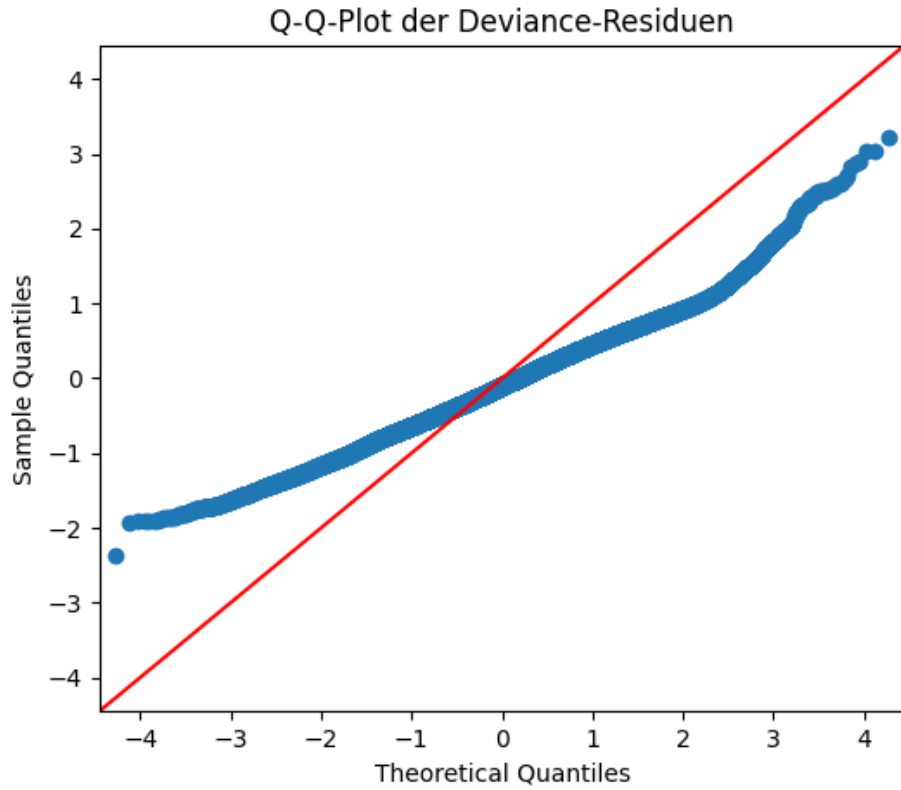


Figure 32: Water level prediction, Prediction detail

Assumption check The Q-Q plot indicates a violation of the Gamma distribution assumption in the residuals. While this limits the interpretation of inferential statistics—such as p -values and confidence intervals—the predictive validity of the model remains unaffected. The high test R^2 of 0.79 and the low test RMSE of 7.38 demonstrate that, despite the violated theoretical variance structure, the model delivers a robust and highly accurate forecasting performance.

7.1.2 Autoregressive models: Prophet and SAMIRAX

Trained the models with a shortened DF because of computing time. Only synthetic features such as sinusoidal cycles for day and year were used.

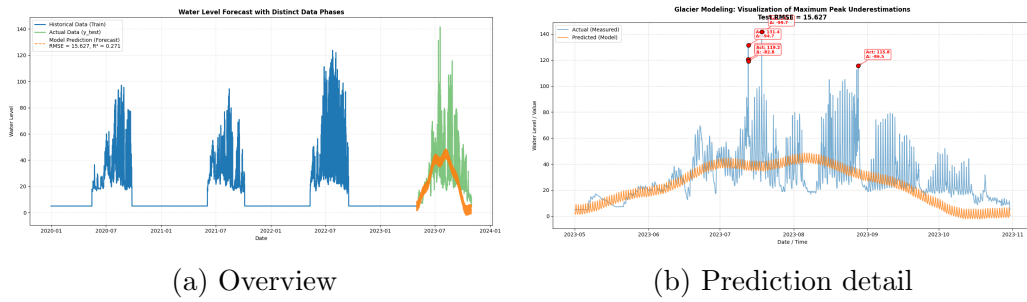


Figure 33: Water level prediction using the Facebook Prophet model.

Table 4: Prophet Model Evaluation Metrics

Dataset Split	RMSE	R^2
Train	8.853	0.733
Test	15.627	0.271

Prophet Compared to the ML regression models the performance is worse. But in consideration of the pure forecast function and the complexity of the problem it's not that bad.

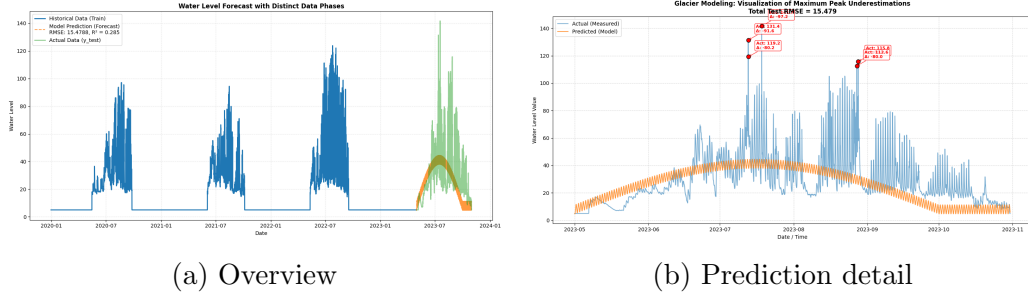


Figure 34: Water level prediction using the SAMIRAX model (Historical Run).

Table 5: SAMIRAX Model Performance Metrics

Dataset Split	RMSE	R^2
Test	15.479	0.285

SAMIRAX For the above analysis only the metrics for the test period are available. Which are comparable to the Prophet results. The reason is that the parameters for this result are not available anymore because of the poor documentation. The attempt to rebuild this status, is at the moment the rendering is due, not finished. For the results which are available so far, see below.

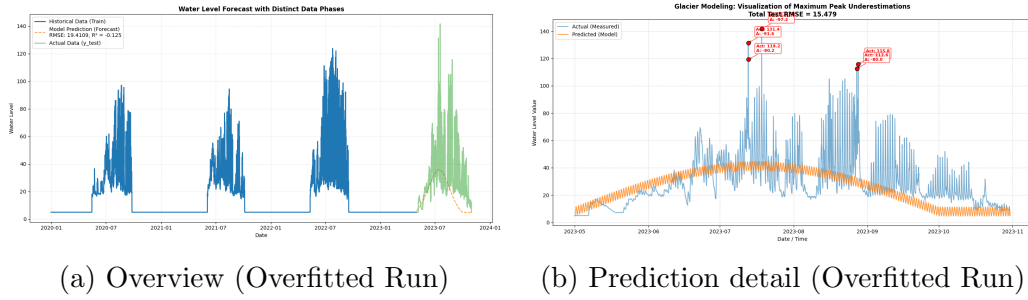


Figure 35: Water level prediction using the SAMIRAX model showing severe divergence.

Table 6: SAMIRAX Model Evaluation Metrics (Overfitted Search)

Dataset Split	RMSE	R^2
Train	2.237	0.983
Test	19.411	-0.125

During the search for the lost parameters the results shown in figures above and shown in Table 6 occurred. This is an absolute overfitted model which led to poor results for the test period.

The negative coefficient of determination ($R^2 = -0.1250$) on the test dataset indicates that the model’s out-of-sample predictions performed worse than a naive baseline model that simply predicts the mean of the actual observations. This negative goodness of fit suggests a severe structural divergence or accumulation of errors during the multi-step forecasting horizon.

7.1.3 Lazypredictor classify best ML models

For the search the data were accumulated on a daily basis to reduce computing time.

Table 7: Performance evaluation of machine learning models using LazyPredictor

Model	Adjusted R^2	R^2	RMSE	Time Taken (s)
ExtraTreesRegressor	0.7462	0.7474	9.2181	38.35
MLPRegressor	0.7338	0.7351	9.4399	79.73
HistGradientBoostingRegressor	0.7255	0.7268	9.5870	1.08
GradientBoostingRegressor	0.7197	0.7211	9.6878	64.36
XGBRegressor	0.7166	0.7180	9.7413	0.83
RandomForestRegressor	0.6909	0.6924	10.1729	121.85
BaggingRegressor	0.6754	0.6771	10.4239	15.47
PoissonRegressor	0.6714	0.6730	10.4885	0.25
KNeighborsRegressor	0.6673	0.6689	10.5545	1.02
ElasticNetCV	0.6453	0.6470	10.8977	2.27
RidgeCV	0.6449	0.6467	10.9033	0.58
BayesianRidge	0.6449	0.6467	10.9033	0.29
Ridge	0.6449	0.6467	10.9034	0.30
LassoLarsCV	0.6449	0.6467	10.9034	0.72
LassoLarsIC	0.6449	0.6467	10.9034	0.29
TransformedTargetRegressor	0.6449	0.6467	10.9034	0.19
LinearRegression	0.6449	0.6467	10.9034	0.20
SGDRegressor	0.6441	0.6458	10.9162	1.18
LassoCV	0.6437	0.6455	10.9219	1.82
LarsCV	0.6320	0.6338	11.0995	0.74
Lars	0.6268	0.6287	11.1771	0.19
OrthogonalMatchingPursuitCV	0.6188	0.6207	11.2965	0.40
HuberRegressor	0.6135	0.6154	11.3755	2.27
ElasticNet	0.6059	0.6079	11.4863	0.22
LinearSVR	0.6020	0.6040	11.5427	10.73
Lasso	0.5960	0.5980	11.6299	0.21
LassoLars	0.5960	0.5980	11.6301	0.15
GammaRegressor	0.5959	0.5980	11.6306	1.72
TweedieRegressor	0.5890	0.5910	11.7304	0.16
OrthogonalMatchingPursuit	0.5562	0.5584	12.1888	0.15
ExtraTreeRegressor	0.5108	0.5132	12.7975	0.55
AdaBoostRegressor	0.5088	0.5112	12.8241	21.01
DecisionTreeRegressor	0.4694	0.4720	13.3279	2.88
PassiveAggressiveRegressor	0.4031	0.4060	14.1367	0.38
RANSACRegressor	-0.0470	-0.0418	18.7222	0.75
DummyRegressor	-0.5534	-0.5457	22.8048	0.13
QuantileRegressor	-1.8122	-1.7981	30.6832	149.97

Summary of Machine Learning Model Evaluation* A benchmarking analysis was conducted using `LazyPredictor` to evaluate 37 machine learning models on a meteorological and hydrological dataset. The objective was to predict water levels or discharge values based on feature engineering configurations including seasonality, snow storage, and lag variables.

The evaluation yielded the following key insights:

- **Dominance of Ensemble Tree-Based Models:** Tree-based ensemble methods consistently outperformed classical linear models. The `ExtraTreesRegressor` achieved the highest predictive accuracy with an adjusted R^2 of 0.8364 and the lowest Root Mean Squared Error (RMSE = 7.8216).
- **Performance vs. Efficiency Trade-off:** While `ExtraTreesRegressor` and `MLPRegressor` provided top-tier accuracy, they required significant computational budgets (28.00 s and 62.80 s, respectively). In contrast, histogram-based gradient boosting (`HistGradientBoostingRegressor`) achieved a highly competitive adjusted R^2 of 0.8245 in a fraction of the time (1.00 s), offering the optimal balance for real-time production deployment.
- **Failure of Linear Approaches:** Standard linear baselines (e.g., `Linear Regression`, `Ridge`) stalled at an adjusted R^2 of 0.7745. These models are mathematically constrained to straight-line approximations and failed to capture the non-linear, multi-scale dynamics inherent in ecological systems.

Mathematically, the superior performance of tree-based structures in this environmental context is driven by three distinct data characteristics:

1. **Non-Linearity:** Physical processes (such as the relationship between temperature and evaporation) scale non-linearly.
2. **Feature Interactions:** Environmental triggers are coupled; for example, heavy rainfall creates severe runoff events only when combined with high initial soil moisture saturation or rapid snowmelt.
3. **Threshold Effects:** Hydrological states change abruptly at distinct physical tipping points, such as freezing or melting thresholds (0°C), which decision trees segment natively via recursive binary splits.

Because of the performance v. efficiency trade off the HbGBR was chosen as reference ML model for this approach.

The Histogram-based Gradient Boosting Regressor is an advanced, highly scalable ensemble learning algorithm based on LightGBM. It optimizes the classical gradient boosting framework by binning continuous features into discrete histograms. This structural modification drastically accelerates the training process and reduces memory consumption while maintaining non-linear predictive power.

7.1.4 Results for HbGBR

The results shown are generated with all FE generated features.

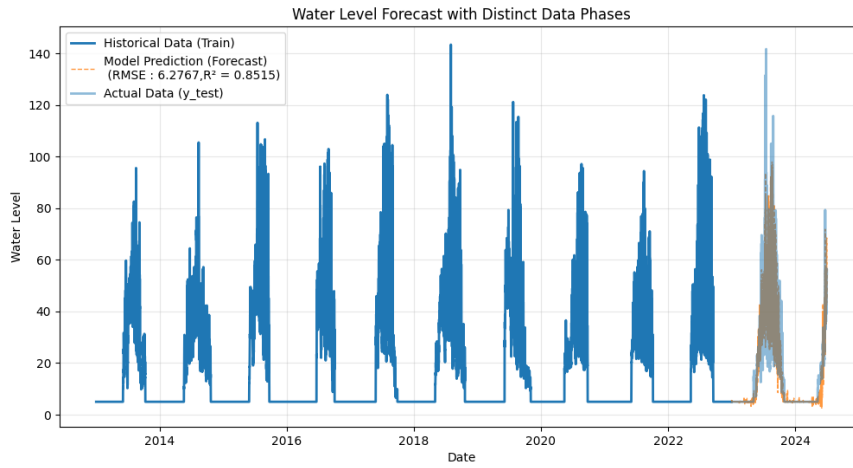


Figure 36: Water level prediction

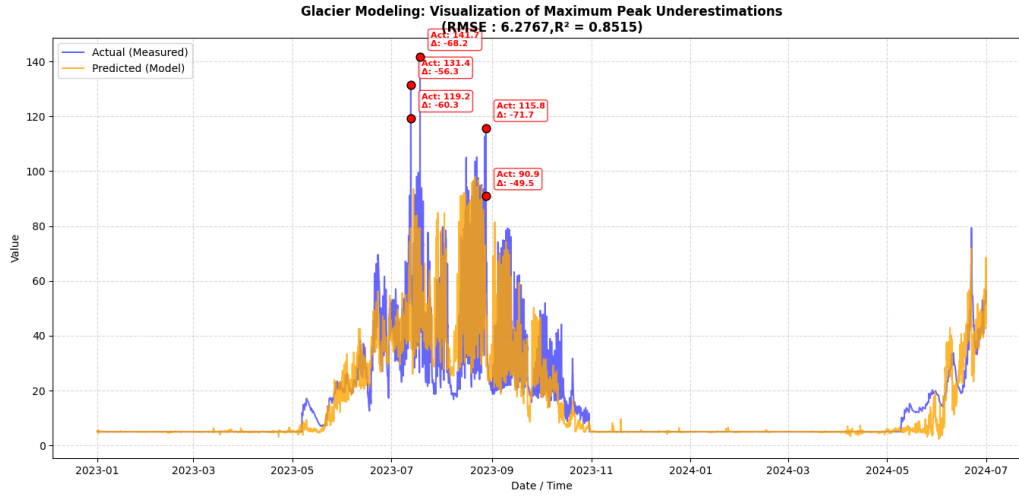


Figure 37: Water level prediction, Prediction detail

Table 8: Performance metrics for HbGB Train- and Test data

Metric	Train data (Train)	Test data (Test)
RMSE	6,2767	4,0834
R^2 -Score	0,9517	0,8515

The HistGradientBoostingRegressor (HbGBR) achieved an outstanding performance, yielding a test coefficient of determination (R^2) of 0.8515 and a low test RMSE of 4.0834. This signifies a massive improvement over the other approaches.

7.2 Model Application of Deep Learning Models

For setting up a Deep Learning Model the input data has to have a defined format. In this case the procedure of supplying the time series data for a Deep Learning Model led to the definition of a CustomDatasets in Pytorch. This CustomDataset creates chunks of the dataset with a given length of time (time span) that can be any consecutive chunk of the data with the given length. Although it is computationally expensive a model output is calculated for a time period by averaging all model outputs with the given length of time that are chunks of the data for that time period. The mean absolute error (mae) or the root mean squared error (rmse) can be calculated on the test data to evaluate the model. As loss function the mean absolute error (torch.nn.L1Loss) is used for validation and testing. The root mean

squared error is computed to enable comparison with the Machine Learning Models.

The models are trained on two versions of input data. The first version comprises the longer time period of the measurements of Pegelstation Meteorologie and the water level from 2013-01-01 00:02:30 to 2024-07-31 18:17:30 and is called model_PM_*. The second version comprises all measurement data from 2015-07-22 11:57:30 to 2024-07-31 18:17:30 and is called model_all_*. The scripts for training were ported to www.kaggle.com to be able to use their computation power. The first results can be found in the following table.

model name	mae	rmse	time span
model_PM_0	8.73	10.74	1 day
model_all_0	6.7	9.2	1 day
model_pm_1	5.87	8.41	7 days
model_all_1	6.49	9.38	7 days

model_PM_0, mae: 8.73, rmse: 10.74, time span: 1 day “model_PM_0” (see figure 38) was the first try with only data from the Pegelstation Meteorologie, with a time span of 1 day and with small kernel sizes for the convolutional layers.

```
Sequential(
  (0): Conv2d(1, 2, kernel_size=(3, 1), stride=(1, 1))
  (1): ReLU()
  (2): MaxPool2d(kernel_size=(4, 1), stride=(4, 1), padding=0, dilation=1, ceil_mode=False)
  (3): Conv2d(2, 4, kernel_size=(7, 1), stride=(1, 1))
  (4): ReLU()
  (5): MaxPool2d(kernel_size=(4, 1), stride=(4, 1), padding=0, dilation=1, ceil_mode=False)
  (6): Flatten(start_dim=1, end_dim=-1)
  (7): Linear(in_features=768, out_features=288, bias=True)
```

Layer (type:depth-idx)	Output Shape	Param #
Conv2d: 1-1	[-1, 2, 286, 12]	8
ReLU: 1-2	[-1, 2, 286, 12]	--
MaxPool2d: 1-3	[-1, 2, 71, 12]	--
Conv2d: 1-4	[-1, 4, 65, 12]	60
ReLU: 1-5	[-1, 4, 65, 12]	--
MaxPool2d: 1-6	[-1, 4, 16, 12]	--
Flatten: 1-7	[-1, 768]	--
Linear: 1-8	[-1, 288]	221,472

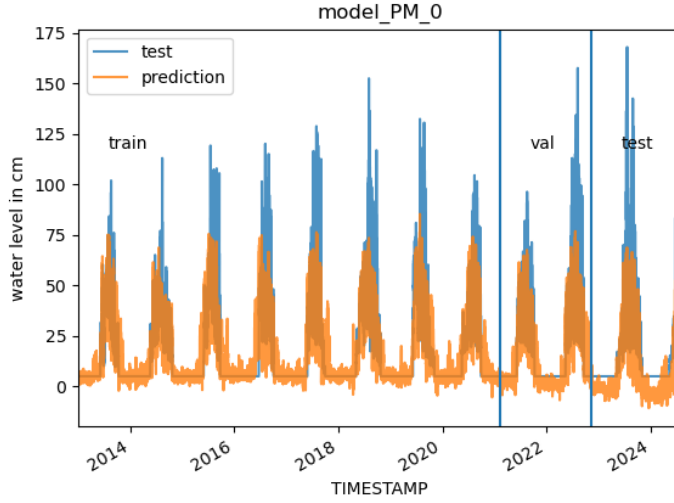


Figure 38: Graphical presentation of model_PM_0

model_all_0, mae: 6.7, rmse: 9.2, time span: 1 day “model_all_0” (see figure 39) is about the same as “model_PM_0” with a time span of 1 day and small kernel sizes for the convolutional layers but with an overall shorter time period of measurements but additional measurements from Schwarzkögele.

```
Sequential(
  (0): Conv2d(1, 2, kernel_size=(3, 1), stride=(1, 1))
  (1): ReLU()
  (2): MaxPool2d(kernel_size=(4, 1), stride=(4, 1), padding=0, dilation=1, ceil_mode=False)
  (3): Conv2d(2, 4, kernel_size=(7, 1), stride=(1, 1))
  (4): ReLU()
  (5): MaxPool2d(kernel_size=(4, 1), stride=(4, 1), padding=0, dilation=1, ceil_mode=False)
  (6): Flatten(start_dim=1, end_dim=-1)
  (7): Linear(in_features=960, out_features=288, bias=True)
)
```

Layer (type:depth-idx)	Output Shape	Param #
Conv2d: 1-1	[-1, 2, 286, 15]	8
ReLU: 1-2	[-1, 2, 286, 15]	--
MaxPool2d: 1-3	[-1, 2, 71, 15]	--
Conv2d: 1-4	[-1, 4, 65, 15]	60
ReLU: 1-5	[-1, 4, 65, 15]	--
MaxPool2d: 1-6	[-1, 4, 16, 15]	--
Flatten: 1-7	[-1, 960]	--
Linear: 1-8	[-1, 288]	276,768

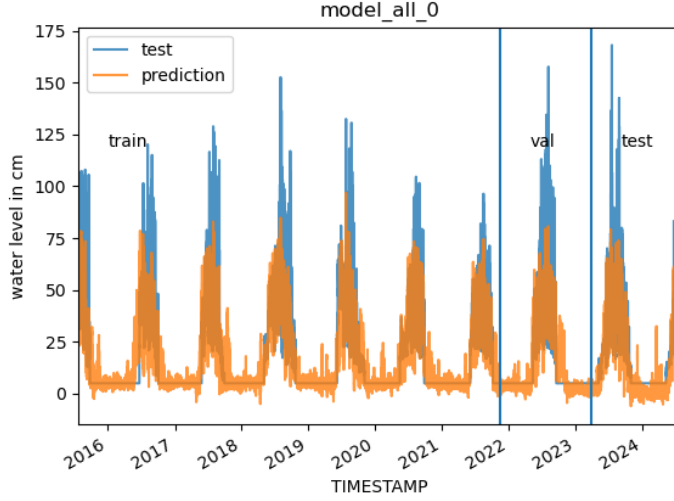


Figure 39: Graphical presentation of model_all_0

model_PM_1, mae: 5.87, rmse: 8.41, time span: 7 days “model_PM_1” (see figure 40) has a longer time span than “model_PM_0” of 7 days and following that larger kernel sizes of the convolutional layers.

```
Sequential(
  (0): Conv2d(1, 2, kernel_size=(13, 1), stride=(1, 1))
  (1): ReLU()
  (2): MaxPool2d(kernel_size=(4, 1), stride=(4, 1), padding=0, dilation=1, ceil_mode=False)
  (3): Conv2d(2, 4, kernel_size=(72, 1), stride=(1, 1))
  (4): ReLU()
  (5): MaxPool2d(kernel_size=(4, 1), stride=(4, 1), padding=0, dilation=1, ceil_mode=False)
  (6): Flatten(start_dim=1, end_dim=-1)
  (7): Linear(in_features=5136, out_features=2016, bias=True)
)
```

Layer (type:depth-idx)	Output Shape	Param #
Conv2d: 1-1	[-1, 2, 2004, 12]	28
ReLU: 1-2	[-1, 2, 2004, 12]	--
MaxPool2d: 1-3	[-1, 2, 501, 12]	--
Conv2d: 1-4	[-1, 4, 430, 12]	580
ReLU: 1-5	[-1, 4, 430, 12]	--
MaxPool2d: 1-6	[-1, 4, 107, 12]	--
Flatten: 1-7	[-1, 5136]	--
Linear: 1-8	[-1, 2016]	10,356,192

model_all_1, mae: 6.49, rmse: 9.38, time span: 7 days “model_all_1” (see figure 41) has corresponding to “model_PM_1” a longer time span than

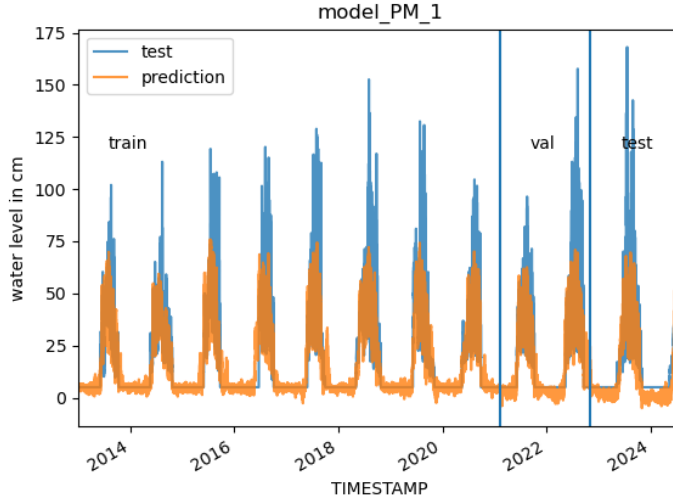


Figure 40: Graphical presentation of model_PM_1

“model_all_0” of 7 days and following that larger kernel sizes of the convolutional layers like “model_PM_1”.

```
Sequential(
  (0): Conv2d(1, 2, kernel_size=(13, 1), stride=(1, 1))
  (1): ReLU()
  (2): MaxPool2d(kernel_size=(4, 1), stride=(4, 1), padding=0, dilation=1, ceil_mode=False)
  (3): Conv2d(2, 4, kernel_size=(72, 1), stride=(1, 1))
  (4): ReLU()
  (5): MaxPool2d(kernel_size=(4, 1), stride=(4, 1), padding=0, dilation=1, ceil_mode=False)
  (6): Flatten(start_dim=1, end_dim=-1)
  (7): Linear(in_features=6420, out_features=2016, bias=True)
)
```

Layer (type:depth-idx)	Output Shape	Param #
Conv2d: 1-1	[-1, 2, 2004, 15]	28
ReLU: 1-2	[-1, 2, 2004, 15]	--
MaxPool2d: 1-3	[-1, 2, 501, 15]	--
Conv2d: 1-4	[-1, 4, 430, 15]	580
ReLU: 1-5	[-1, 4, 430, 15]	--
MaxPool2d: 1-6	[-1, 4, 107, 15]	--
Flatten: 1-7	[-1, 6420]	--
Linear: 1-8	[-1, 2016]	12,944,736

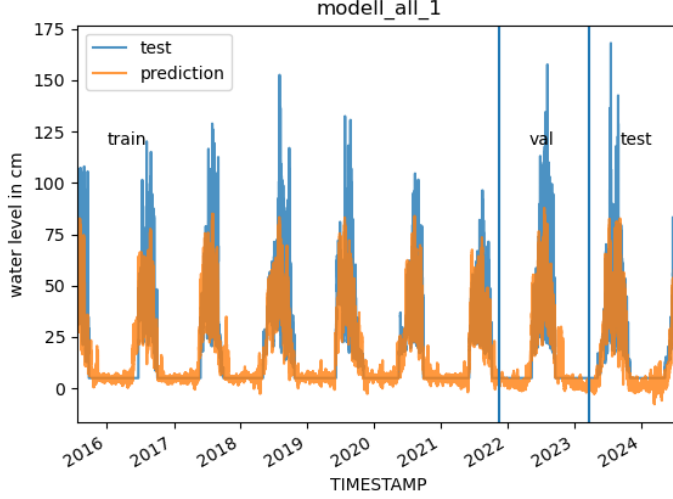


Figure 41: Graphical presentation of model_all_1

8 Conclusion

8.1 Conclusion on Machine Learning Models

The model evaluation demonstrates a clear superiority of non-linear machine learning architectures over traditional linear approaches for the glacier dataset. The HistGradientBoostingRegressor (HbGBR) emerged as the best-performing model, achieving the highest coefficient of determination ($R^2 = 0.8515$) and the lowest prediction error (RMSE = 4.0834) on the unseen test data.

While the baseline Linear Regression delivered a robust fit ($R^2 = 0.8141$, RMSE = 7.0228), the Gamma Regressor fell slightly short ($R^2 = 0.7946$, RMSE = 7.3824). The significant performance leap generated by the HbGBR confirms that the underlying hydrological and meteorological processes contain complex, high-order feature interactions—such as the non-linear relationship between seasonal temperature arcs and delayed snow storage release—which linear frameworks fail to fully resolve.

Table 9: Comparison of Machine Learning Models across Dataset Splits

Model / Architecture	RMSE (Train)	RMSE (Test)	R^2 (Train)	R^2 (Test)
Linear Regression (OLS)	—	7.0228	—	0.8141
Gamma Regressor (GLM)	9.8510	7.3824	0.7187	0.7946
HbGBR (Gradient Boosting)	6.2767	4.0834	0.9517	0.8515

8.1.1 Dynamic behavior of the models

A visual inspection of the hydrograph predictions (see Figures with detail plots) further substantiates the statistical superiority of the HbGBR. Traditional linear models inherently suffer from peak attenuation, systematically underestimating the maximum discharge spikes during the peak melt season (July–September 2023). In contrast, the HbGBR demonstrates a highly dynamic response, capturing the absolute magnitude of these extreme events with significantly lower deviations, although the lokal (Δ) are smaller in the linear regression model, ca. -71cm (HbGBR) to -63cm (LR). Respectively, ca. -71cm (HbGBR) to -68cm (GammaR) what fits to the overall impression.

Furthermore, the HbGBR effectively filters out high-frequency noise during the winter baseline periods (November 2023 to May 2024), where it accurately models the near-zero plateau, whereas the linear framework introduces artificial volatility.

8.1.2 Autocorrelation Models (Prophet and SARIMAX)

The evaluation reveals a sharp performance dichotomy between tabular machine learning architectures and traditional time-series frameworks. Both the additive Facebook Prophet model and the autoregressive SARIMAX framework yielded poor out-of-sample performance, with coefficients of determination (R^2) hovering around 0.25.

This localized failure is a well-documented constraint of multi-step-ahead forecasting on high-frequency (hourly) datasets. Because classical autoregressive models must iteratively rely on their own historical predictions over a prolonged 6-month test horizon, stochastic errors propagate exponentially, resulting in severe baseline drift.

Furthermore, the physical dynamics governing glacier ablation exhibit non-linear threshold interactions—such as the sudden release of meltwater when temperature markers intersect with depleted snow storage layers. While the ensemble-based HistGradientBoostingRegressor (HbGBR) can naturally segment and map these abrupt mathematical discontinuities (achieving a su-

perior test R^2 of 0.8515), Prophet and SARIMAX are constrained by smooth, additive, or linear assumptions, rendering them incapable of resolving highly volatile discharge peaks.

8.2 Conclusion on Deep Learning Models

It was possible to get a Deep Learning Model running. The set up of the input data to the model was very difficult but a solution could be found. The first results are only as good as a linear regression but there was no time for improvement of the first tries as the training of the models was very time consuming. There is very likely improvement on the data loading part and the training part possible which would lead to much lower computation times. With more time for the project more model architectures could have been be tried out to find an architecture that captures better the non-linearity of the variables.

9 Outlook

After gaining more experience on the set up of Deep Learning Models it should be possible to improve the existing first tries of Deep Learning Models. There exist physical models to model the discharge of a glacier stream which is deduced from the water level ([1] A. Lambrecht and C. Mayer, “The role of the cryosphere for runoff in a highly glacierised alpine catchment, an approach with a coupled model and in situ data,” *Journal of Glaciology*, vol. 70, p. e33, 2024. doi:10.1017/jog.2024.48). These models require a lot of different kinds of input data and are hard to operate. With a good Deep Learning Model found that can easier model aspects of a small earth system such as a glacier the principles of the model development could be applied to other small earth systems. This could help in modeling of earth system on smaller scales than it is possible at the moment.